

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 13

Semaphores in Your Own Kernel (xv6)

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. The numbers and the user side **2**
2.1 Check 1: the calls reach the kernel, which does not know them yet 4

3 Part 2. The kernel side **5**
3.1 `kernel/sem.c` 5
3.2 Compiling `sem.c` and connecting the calls 6
3.3 How `sleep_prepare`, `sleep` and `wakeup` work in this kernel 7
3.4 Check 2: `semtest` 7

4 Part 3. A semaphore as a lock: `mix` **8**
4.1 Observations 9

5 Optional. `sem_trydown` **10**

6 Conclusion **11**

1 Goal of the work

In Labs 11 and 12 semaphores were *used* from the C library. In this laboratory work they are *built* inside the xv6 kernel, as three new system calls: `sem_init`, `sem_down` and `sem_up`, with the numbers 24, 25 and 26. The work shows that a semaphore is only a number, a spinlock that protects the number, and the `sleep`/`wakeup` mechanism of the kernel. A process that waits on a semaphore really goes to sleep and uses no CPU.

The work started from the xv6 tree of Lab 10, which already contains the `trace` system call (number 23). It was built with `riscv64-linux-gnu-gcc` 13.3.0 and run with `make qemu` in QEMU 8.2.2 with 3 CPUs, on Ubuntu 24.04 in a Docker container on macOS. The modified tree is in `lab13/xv6`.

2 Part 1. The numbers and the user side

The recipe is the same as for `trace` in Lab 10, applied three times. The semaphores are numbered 0 to 7 and are shared by all processes.

Listing 1: `kernel/syscall.h`: the new numbers

```
#define SYS_sync   22
#define SYS_trace 23
#define SYS_sem_init 24  // new
#define SYS_sem_down 25  // new
#define SYS_sem_up   26  // new
```

Listing 2: `user/user.h`: the declarations

```
int sync(void);
int trace(int);
int sem_init(int, int);   // new: id, start value
int sem_down(int);        // new: take one, or sleep
int sem_up(int);          // new: give one back
```

Listing 3: `user/usys.pl`: the user-space stubs

```
entry("sync");
entry("trace");
entry("sem_init");  # new
entry("sem_down");  # new
entry("sem_up");    # new
```

The test program `semtest` has two tests. In the first one, the child must wait for the parent, because semaphore 0 starts at 0. In the second one, ping and pong take turns using two semaphores that pass the turn to each other.

Listing 4: `user/semtest.c`

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
```

```
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

Listing 5: `user/mix.c`

```
// mix.c: two processes print five lines each, one letter per write().
//   mix         no lock: the letters of the two processes can mix
//   mix lock    semaphore 3 used as a lock: every line stays whole
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
```

```
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

Both programs were added to `UPROGS` in the `Makefile`, after `$U/_zombie\`:

```
        $U/_zombie\
        $U/_semtest\
        $U/_mix\
```

2.1 Check 1: the calls reach the kernel, which does not know them yet

At this point the user side exists, but the kernel has no handler for numbers 24–26. The kernel prints an error and returns -1 for every such call:

Listing 6: Output of `semtest` after Part 1

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
3 semtest: unknown sys call 26
3 semtest: unknown sys call 24
3 semtest: unknown sys call 24
3 semtest: unknown sys call 25
5 semtest: unknown sys call 25
p i npgo ng1
13 semtest: unknown sys call 26
3 semtest: unknown sys call 25

p5 semtest: unknown sys call 26
5 semtest: unknown sys call 25
 i npong 2
5 semtest: unknowng sys call 26
5 semtest: unknown sys call 25
  2
p3 semtest: unknown sys call 26
o3 semtest: unknown sys call 25
ng p3i
n5 semtest: unknown sys call 26
g 3
3 semtest: unknown sys call 26
semtest: OK
```

The first lines are exactly the expected ones. The line `child:` comes **before** the line `parent:`, because `sem_down(0)` did nothing, so the child did not wait for the parent. In the second test, ping and pong are not synchronized either, and their letters mix with the error messages of the kernel.

3 Part 2. The kernel side

3.1 `kernel/sem.c`

The new file keeps a table of 8 semaphores. Each one is a counter together with a spinlock that protects it. `sem_down` is the central part. While the value is 0, the process registers the address of the semaphore as the event it waits for (`sleep_prepare`), releases the spinlock, and sleeps. `sem_up` adds 1 and wakes every process that sleeps on that address.

Listing 7: `kernel/sem.c`

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 13).
// A small table of NSEM semaphores, shared by every process.
// Each one is a number plus a spinlock that protects the number.

#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;   // a zeroed spinlock is a free spinlock
  int value;              // how many may still pass
};

static struct sem sems[NSEM];

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep until there is.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);        // I will sleep on the address of s
```

```
    release(&s->lock);       // never sleep holding a spinlock
    sleep();                 // returns at once if sem_up came in between
    acquire(&s->lock);
    if (killed(myproc())) {  // kill must still work on a waiting process
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  s->value++;
  wakeup(s);
  release(&s->lock);
  return 0;
}
```

3.2 Compiling `sem.c` and connecting the calls

`sem.o` was added to the `OBJS` list of the `Makefile`, after `$K/sysproc.o`. The three handlers were added to the system call table in `kernel/syscall.c`. Because this tree also contains `trace` from Lab 10, the names were added to `syscallnames[]` too, so that `trace` can print them.

Listing 8: `Makefile`: `OBJS`

```
  $K/sysproc.o \
  $K/sem.o \
```

Listing 9: `kernel/syscall.c`: the new entries

```
extern uint64 sys_trace(void);
extern uint64 sys_sem_init(void);  // new
extern uint64 sys_sem_down(void);  // new
extern uint64 sys_sem_up(void);    // new
...
  [SYS_trace]    = sys_trace,
  [SYS_sem_init] = sys_sem_init,  // new
  [SYS_sem_down] = sys_sem_down,  // new
  [SYS_sem_up]   = sys_sem_up,    // new
...
  [SYS_trace]  = "trace",
  [SYS_sem_init] = "sem_init",
  [SYS_sem_down] = "sem_down",
```

```
  [SYS_sem_up]   = "sem_up",
```

3.3 How `sleep_prepare`, `sleep` and `wakeup` work in this kernel

This version of xv6 sleeps in two steps, not with `sleep(chan, lock)` as in the xv6 book. `sleep_prepare(chan)` only sets `p->chan = chan`. `sleep()` goes to sleep only if `p->chan` is still set. `wakeup(chan)` clears `p->chan` for every process waiting on `chan` and makes the sleeping ones `RUNNABLE`:

Listing 10: From `kernel/proc.c`

```
void
sleep(void)
{
  struct proc *p = myproc();

  acquire(&p->lock);
  if (p->chan != 0) {
    p->state = SLEEPING;
    sched();
  }
  release(&p->lock);
}

void
wakeup(void *chan)
{
  struct proc *p;

  for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->chan == chan) {
      p->chan = 0;
      if (p->state == SLEEPING) {
        p->state = RUNNABLE;
      }
    }
    release(&p->lock);
  }
}
```

Because of this, a `sem_up` that comes between `release(&s->lock)` and `sleep()` in `sem_down` is not lost: it clears `p->chan`, and `sleep()` returns at once.

3.4 Check 2: `semtest`

Listing 11: Output of `semtest` after Part 2

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
```

```
ping 3
  pong 3
semtest: OK
```

The parent waits 10 ticks (`pause(10)`) before it calls `sem_up(0)`, yet the child prints second: during that time it was asleep in `sem_down(0)`. Ping and pong alternate strictly, three times each, and the program ends with `semtest: OK`. The test gave the same result every time I ran it.

4 Part 3. A semaphore as a lock: `mix`

`mix` starts two processes. Each one prints five lines, with one `write()` system call per letter. `make qemu` starts xv6 with 3 CPUs, so the two processes really run at the same time. The program was run twice without the lock, then once with the lock (semaphore 3, started at 1).

Listing 12: `mix` without the lock, first run

```
$ mix
AAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAb
bA
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbbAbAbAbAbAbAbAAbAb
bA
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Listing 13: `mix` without the lock, second run

```
$ mix
AAAAAAAAAAbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbAbAbAbAbbAbAbAbAbAbAbAbAbAbA
AbAbAbAbAb
bAbAbAbAbAbAbAbAbAbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

Listing 14: `mix lock`

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

```
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Without the lock the two runs are mixed differently. With the lock every line is whole.

4.1 Observations

Question 1. Without the lock, why do the letters mix? How many system calls does one line cost? One line costs **31 system calls**: `lines()` calls `write(1, p, 1)` once for every character, 30 letters plus the `'\n'`. The kernel makes each single `write()` whole, but nothing keeps the 31 writes of one line together. With 3 CPUs the two processes really run at the same time on different CPUs, and the scheduler can also switch between them after any system call. Between two writes of process A, process B can do one or more writes of its own, so the letters of the two lines interleave (`AbAbAb...`). The interleaving depends on timing, so it changes from run to run.

Question 2. With the lock every line is whole. Which two lines of `mix.c` make one line a critical section?

```
  if (uselock)
    sem_down(3);        // before the loop that writes the line
  ...
  if (uselock)
    sem_up(3);          // after the loop
```

Semaphore 3 starts at 1 (`sem_init(3, 1)` in `main`), so it works as a lock. `sem_down(3)` lets only one process at a time into the loop of 31 writes, and `sem_up(3)` lets the other process in only after the whole line, including `'\n'`, has been written.

Question 3. In `sem_down` the test is `while (s->value == 0)`, not `if`. Why can a process wake up and still find 0?

- `sem_up` calls `wakeup(s)`, and `wakeup` wakes **every** process that sleeps on the address `s`, not just one, while `sem_up` added only one unit. The first woken process to take `s->lock` takes the unit, the value is 0 again, and the others wake up and find 0.
- Between `wakeup()` and the moment the woken process runs again and takes `s->lock`, a third process that was not sleeping can call `sem_down`, find the value 1 and take the unit first.
- A process can also leave `sleep()` for a reason that has nothing to do with the semaphore: `kkill()` makes a `SLEEPING` process `RUNNABLE` so that it can notice it was killed. This is why `sem_down` checks `killed()` after waking up.

With `while`, the process checks the value again after every wakeup and goes back to sleep if it is still 0. With `if`, it would execute `s->value-` on 0, the value would become -1, and two processes could be in the critical section at the same time.

Question 4. `sem_down` calls `release(&s->lock)` before `sleep()`. What would go wrong if it slept holding the spinlock? The process that must wake it up could never run `sem_up`. `sem_up` starts with `acquire(&s->lock)`, which would spin forever on another CPU, because the sleeper holds that lock and will not release it until it is woken. The sleeper waits for `sem_up` and `sem_up` waits for the sleeper: a **deadlock**. Every later

`sem_init`, `sem_down` or `sem_up` on that semaphore would also spin forever. In addition, a spinlock turns interrupts off on its CPU (`push_off`), and xv6 refuses to switch away while holding any lock other than `p->lock`: `sched()` checks `mycpu()->noff != 1`, and the kernel would panic with `"sched locks"`.

Releasing the lock before `sleep()` does not lose a wakeup. `sleep_prepare(s)` has already set `p->chan = s`, so if `sem_up` runs between `release()` and `sleep()`, `wakeup(s)` clears `p->chan`, and `sleep()` sees `p->chan == 0` and returns at once instead of sleeping.

5 Optional. `sem_trydown`

A fourth system call, `sem_trydown(id)` with number 27, works like `sem_down`, but if the value is 0 it returns -1 at once instead of sleeping. It was added in the same way as the other three: the number in `syscall.h`, the declaration in `user.h`, the stub in `usys.pl`, the entries in `syscall.c`, and the handler in `sem.c`:

Listing 15: `sys_sem_trydown` in `kernel/sem.c`

```
uint64
sys_sem_trydown(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  if (s->value == 0) {
    release(&s->lock);
    return -1;                // nothing to take: do not wait
  }
  s->value--;
  release(&s->lock);
  return 0;
}
```

Listing 16: `user/trytest.c`

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  sem_init(4, 1);
  printf("value 1: sem_trydown(4) = %d   (took the unit)\n", sem_trydown(4));
  printf("value 0: sem_trydown(4) = %d  (nothing to take, did not sleep)\n",
    sem_trydown(4));
  sem_up(4);
  printf("after sem_up(4): sem_trydown(4) = %d\n", sem_trydown(4));
  exit(0);
}
```

Listing 17: Output of `trytest`

```
$ trytest
```

```
value 1: sem_trydown(4) = 0   (took the unit)
value 0: sem_trydown(4) = -1  (nothing to take, did not sleep)
after sem_up(4): sem_trydown(4) = 0
```

6 Conclusion

A semaphore inside the kernel needs very little: a number, a spinlock that protects the number, and the `sleep`/`wakeup` mechanism of the scheduler. `sem_down` sleeps on the address of the semaphore while the value is 0, and `sem_up` increments it and wakes the sleepers. A waiting process is `SLEEPING` and uses no CPU, unlike a spinlock, which keeps spinning.

Before the kernel side existed, `semtest` ran in the wrong order and printed "unknown sys call". After it, the child waited for the parent, ping and pong alternated strictly, and `mix lock` kept every line whole, while `mix` without the lock mixed the letters differently on every run. The two details that make the implementation correct are the `while` loop, which checks the value again after every wakeup, and releasing the spinlock before sleeping, which together with `sleep_prepare` avoids both deadlocks and lost wakeups.